
PROGRAMME EXPERT EN INGÉNIERIE DES DONNÉES — SEMESTRE 8

ECHE834 — Solutions d'échanges inter-applicatifs

Travaux Dirigés — TD2

Flask vs FastAPI & Architecture RESTful

Choisir son framework, concevoir ses ressources

Durée 3 heures (+ travail autonome)

Prérequis TD1 réalisé (environnement Python, API REST Flask de base)

Session associée Session 2 — Développement et Implémentation d'APIs

Table des matières

Table des matières	2
1. Introduction et objectifs pédagogiques	4
1.1 Où se situe ce TD ?	4
1.2 Objectifs pédagogiques	4
1.3 Compétences visées (référentiel RNCP 36921).....	4
1.4 Organisation du TD	4
2. Rappel : environnement de travail	5
2.1 Réactiver l'environnement	5
2.2 Nouvelles dépendances pour ce TD	5
2.3 Structure de projet pour ce TD	5
3. Exercice 1 — Flask vs FastAPI sur un même cas d'usage	6
3.1 Principe de l'exercice.....	6
3.2 Version Flask — fil conducteur.....	6
3.3 Version FastAPI — même ressource	7
3.4 Mesurer et comparer	7
3.5 Pour la documentation automatique.....	8
4. Exercice 2 — Concevoir une architecture RESTful	9
4.1 Contexte du projet	9
4.2 Étape 1 — Modéliser les ressources (sur papier ou en commentaire).....	9
4.3 Étape 2 — Choisir son framework.....	9
4.4 Étape 3 — Implémenter	9
4.5 Étape 4 — Valider et gérer les erreurs.....	10
4.6 Étape 5 — Tester	10
5. Guide technique — Bonnes pratiques RESTful	11
5.1 Grille de décision : Flask ou FastAPI ?	11
5.2 Nommer ses ressources	11
5.3 Choisir le bon code de statut	11
5.4 Structurer une réponse d'erreur cohérente	11
5.5 Erreurs fréquentes à éviter	12
6. Critères d'évaluation	13
6.1 Exercice 1 — Benchmark Flask vs FastAPI.....	13
6.2 Exercice 2 — Architecture RESTful	13
7. Annexes	14
7.1 Ressources complémentaires	14
7.2 Commandes de référence rapide.....	14

7.3 Troubleshooting	14
---------------------------	----

1. Introduction et objectifs pédagogiques

1.1 Où se situe ce TD ?

Ce second TD prolonge directement le TD1 (création d'une API REST CRUD avec Flask). Vous y aviez construit une API de gestion de livres fonctionnelle, authentifiée par token, validée et testée. Ce TD2 reprend ces acquis pour aller plus loin sur deux axes du programme de la Session 2 : le choix du framework (Flask ou FastAPI) et la conception rigoureuse d'une architecture RESTful.

Acquis du TD1 mobilisés ici

Environnement virtuel Python, structure de projet, routes Flask avec méthodes HTTP, validation manuelle des données, gestion des erreurs via `abort()`, authentification par token Bearer, tests avec `pytest`.

1.2 Objectifs pédagogiques

- Comparer concrètement Flask et FastAPI sur un même cas d'usage, au-delà de la théorie
- Mesurer l'impact de la validation automatique (Pydantic) par rapport à la validation manuelle
- Concevoir une architecture RESTful propre à partir d'un cahier des charges, sans exemple à copier
- Justifier ses choix de structuration d'URL, de méthodes HTTP et de codes de statut
- Argumenter le choix d'un framework selon le contexte d'un projet

1.3 Compétences visées (référentiel RNCP 36921)

- Concevoir l'architecture d'un système d'échange de données inter-applicatif
- Développer une API conforme aux standards REST
- Choisir les technologies adaptées à un contexte projet donné

1.4 Organisation du TD

Section	Durée	Contenu
Exercice 1	~1h15	Benchmark Flask vs FastAPI sur une même ressource
Exercice 2	~1h15	Conception et implémentation d'une architecture RESTful complète
Guide technique	~20 min	Bonnes pratiques de structuration d'API REST
Synthèse	~10 min	Grille de décision Flask / FastAPI

2. Rappel : environnement de travail

L'environnement créé lors du TD1 (dossier `td_api_eche834`, environnement virtuel `venv`) est réutilisé tel quel. Si besoin, voici les commandes essentielles pour le réactiver.

2.1 Réactiver l'environnement

```
cd td_api_eche834
source venv/bin/activate    # Linux/macOS
.\venv\Scripts\Activate.ps1 # Windows PowerShell

# Vérifier les paquets déjà installés (TD1)
pip list
```

2.2 Nouvelles dépendances pour ce TD

Aucune nouvelle dépendance majeure n'est requise : `flask`, `fastapi`, `uvicorn`, `pydantic` et `pytest` ont déjà été installés au TD1. Vérifiez simplement leur présence :

```
pip show flask fastapi uvicorn pydantic pytest

# Si l'un d'eux manque :
pip install flask fastapi 'uvicorn[standard]' pydantic pytest httpx
```

2.3 Structure de projet pour ce TD

Créez deux nouveaux dossiers à la racine de votre projet existant, dédiés à cet exercice :

```
td_api_eche834/
├── venv/
├── exercice1_rest/          # déjà créé au TD1
├── td2_benchmark/          # NOUVEAU – Exercice 1
│   ├── version_flask/
│   │   └── app.py
│   ├── version_fastapi/
│   │   └── app.py
├── td2_architecture/       # NOUVEAU – Exercice 2
│   ├── app.py
│   ├── tests/
│   │   └── test_api.py
└── requirements.txt
```

3. Exercice 1 — Flask vs FastAPI sur un même cas d'usage

3.1 Principe de l'exercice

Plutôt que comparer Flask et FastAPI sur le papier, vous allez implémenter la même ressource dans les deux frameworks et observer concrètement les différences de code, de validation et de comportement. La ressource à exposer est un carnet de contacts.

Ressource à implémenter : /contacts

Chaque contact possède : nom (texte), email (texte), telephone (texte), favori (booléen). Les deux versions (Flask et FastAPI) doivent exposer exactement les mêmes endpoints fonctionnels, pour permettre une comparaison loyale.

3.2 Version Flask — fil conducteur

Dans `td2_benchmark/version_flask/app.py`, reproduisez la structure vue au TD1 (middleware de token, validation manuelle, gestion d'erreurs) appliquée cette fois à /contacts. Voici le squelette de départ à compléter :

```
#!/usr/bin/env python3
"""API REST - Carnet de contacts (Flask)"""
from flask import Flask, jsonify, request, abort

app = Flask(__name__)
API_TOKEN = "mon_token_secret_123"

contacts_db = [
    {"id": 1, "nom": "Aïcha Diallo", "email": "aicha@exemple.fr",
     "telephone": "0601020304", "favori": True},
]
next_id = 2

def verifier_token():
    # À compléter : même logique qu'au TD1
    ...

def valider_contact(data, required_fields=None):
    # À compléter : nom et email obligatoires,
    # email doit contenir '@', telephone optionnel
    ...

# TODO : routes GET /contacts (liste + filtre par favori)
# TODO : route GET /contacts/<id>
# TODO : route POST /contacts
# TODO : route PUT /contacts/<id>
# TODO : route DELETE /contacts/<id>
```

Consigne : complétez ce squelette pour obtenir une API Flask fonctionnelle et fidèle à ce que vous avez déjà produit au TD1, mais sur la ressource /contacts.

3.3 Version FastAPI — même ressource

Dans `td2_benchmark/version_fastapi/app.py`, implémentez exactement les mêmes endpoints avec FastAPI et Pydantic. Voici le squelette de départ :

```
#!/usr/bin/env python3
"""API REST - Carnet de contacts (FastAPI)"""
from fastapi import FastAPI, HTTPException, Header, Query
from pydantic import BaseModel, EmailStr
from typing import Optional, List

app = FastAPI(title="API Contacts")
API_TOKEN = "mon_token_secret_123"

class Contact(BaseModel):
    id: int
    nom: str
    email: EmailStr
    telephone: Optional[str] = None
    favori: bool = False

class ContactInput(BaseModel):
    nom: str
    email: EmailStr
    telephone: Optional[str] = None
    favori: bool = False

contacts_db: List[dict] = [
    {"id": 1, "nom": "Aïcha Diallo", "email": "aicha@exemple.fr",
     "telephone": "0601020304", "favori": True},
]
next_id = 2

def verifier_token(authorization: str = Header(...)):
    # À compléter : même logique de vérification que côté Flask
    ...

# TODO : endpoints équivalents à la version Flask,
# en s'appuyant sur la validation automatique de Pydantic
```

Notez ce que Pydantic prend déjà en charge automatiquement (que vous deviez écrire à la main côté Flask) : typage des champs, validation du format email avec `EmailStr`, valeur par défaut pour `favori`.

3.4 Mesurer et comparer

Une fois les deux versions fonctionnelles, complétez le tableau comparatif suivant à partir de votre propre expérience de développement :

Critère observé	Flask	FastAPI
Nombre de lignes pour valider un contact	à mesurer	à mesurer
Erreur retournée si email mal formé	à tester	à tester
Documentation générée automatiquement	à tester	à tester
Temps de mise en place ressenti	à évaluer	à évaluer

Pour le champ « Erreur retournée si email mal formé », testez en envoyant un contact avec "email": "pas-un-email" sur les deux versions et comparez la réponse HTTP obtenue (code de statut et contenu du message d'erreur).

```
curl -X POST http://localhost:5000/contacts \
  -H "Authorization: Bearer mon_token_secret_123" \
  -H "Content-Type: application/json" \
  -d '{"nom": "Test", "email": "pas-un-email"}'
```

3.5 Pour la documentation automatique

Lancez la version FastAPI et ouvrez <http://localhost:8000/docs> dans votre navigateur. Comparez cette documentation interactive à ce que vous aviez dû produire manuellement (ou pas du tout) côté Flask au TD1.

4. Exercice 2 — Concevoir une architecture RESTful

4.1 Contexte du projet

Une médiathèque municipale vous confie la conception de l'API qui gérera ses réservations de salles. Contrairement au TD1 où la structure était donnée, vous devez ici concevoir vous-même l'architecture à partir du cahier des charges ci-dessous, avant d'écrire la première ligne de code.

Cahier des charges

La médiathèque dispose de plusieurs salles (nom, capacité, équipement). Les usagers peuvent réserver une salle pour un créneau donné (date, heure de début, heure de fin, nom de l'utilisateur). Une salle ne peut pas être réservée deux fois sur un créneau qui se chevauche. L'API doit permettre de consulter les salles, consulter les réservations d'une salle, créer une réservation, et annuler une réservation.

4.2 Étape 1 — Modéliser les ressources (sur papier ou en commentaire)

Avant de coder, répondez à ces questions de conception. Notez vos réponses en commentaire en tête de votre fichier app.py :

1. Quelles sont les ressources principales de ce système ? Combien y en a-t-il ?
2. Une réservation est-elle une ressource indépendante, ou une sous-ressource d'une salle ? Justifiez.
3. Quelle URL utiliseriez-vous pour : lister toutes les salles ? Lister les réservations d'une salle précise ? Créer une réservation ?
4. Quelle méthode HTTP et quel code de statut pour annuler une réservation ?

4.3 Étape 2 — Choisir son framework

Choisissez Flask ou FastAPI pour cet exercice, et justifiez votre choix en une phrase en commentaire dans le code (en vous appuyant sur la grille de décision de la section 5).

4.4 Étape 3 — Implémenter

Dans `td2_architecture/app.py`, implémentez votre architecture. Le squelette ci-dessous illustre la structure attendue (à adapter selon votre framework choisi) :

```
salles_db = [
    {"id": 1, "nom": "Salle Voltaire", "capacite": 12,
     "equipement": ["vidéoprojecteur", "tableau blanc"]},
    {"id": 2, "nom": "Salle Curie", "capacite": 6,
     "equipement": ["écran TV"]},
]

reservations_db = [
    {"id": 1, "salle_id": 1, "usager": "M. Dupont",
     "date": "2026-06-20", "heure_debut": "14:00", "heure_fin": "15:30"},
]

# TODO : GET /salles
# TODO : GET /salles/<id>/reservations
```

```
# TODO : POST /salles/<id>/reservations
#         -> doit vérifier l'absence de chevauchement de créneau
# TODO : DELETE /reservations/<id>
```

Le contrôle de chevauchement de créneau est le point le plus délicat de cet exercice : deux réservations sur la même salle et la même date ne doivent pas avoir d'heures qui se superposent. Si une tentative de réservation chevauche un créneau existant, l'API doit refuser la création avec un code 409 Conflict.

Rappel sur le code 409

409 Conflict signale que la requête est syntaxiquement valide mais entre en conflit avec l'état actuel de la ressource. C'est le code approprié ici, à distinguer du 400 (donnée mal formée) et du 404 (ressource introuvable).

4.5 Étape 4 — Valider et gérer les erreurs

Votre API doit gérer correctement les cas suivants, avec le code de statut adapté à chacun :

Situation	Code attendu
Réservation créée avec succès	201 Created
Champ date, heure_debut ou usager manquant	400 Bad Request
Salle inexistante référencée	404 Not Found
Créneau qui chevauche une réservation existante	409 Conflict
Suppression d'une réservation réussie	204 No Content
Token absent ou invalide	401 / 403

4.6 Étape 5 — Tester

Dans `td2_architecture/tests/test_api.py`, écrivez au minimum les tests suivants (sur le modèle de ceux du TD1) :

- Lister les salles renvoie un statut 200 et une liste non vide
- Créer une réservation valide renvoie 201
- Créer une réservation sur un créneau déjà pris renvoie 409
- Créer une réservation sur une salle inexistante renvoie 404
- Annuler une réservation existante renvoie 204

5. Guide technique — Bonnes pratiques RESTful

5.1 Grille de décision : Flask ou FastAPI ?

Utilisez cette grille pour justifier votre choix de framework, dans cet exercice comme dans vos futurs projets.

Si votre projet présente...	Framework conseillé
Un prototype à livrer très rapidement	Flask
Une équipe qui maîtrise déjà l'un des deux	Celui maîtrisé
Un fort besoin de validation stricte des données	FastAPI
Une exigence contractuelle de documentation Swagger	FastAPI
Un trafic élevé ou des appels I/O concurrents	FastAPI
Une architecture déjà construite sur Flask (legacy)	Flask

5.2 Nommer ses ressources

- Utiliser des noms (substantifs), jamais des verbes : /reservations et non /reserverSalle
- Toujours au pluriel pour une collection : /salles, pas /salle
- Les sous-ressources s'expriment par imbrication : /salles/3/reservations
- Éviter plus de deux niveaux d'imbrication : préférer /reservations/12 à /salles/3/reservations/12 une fois l'identifiant connu

5.3 Choisir le bon code de statut

Code	Signification	Cas d'usage typique
200	OK	Lecture ou mise à jour réussie
201	Created	Création réussie (POST)
204	No Content	Suppression réussie
400	Bad Request	Donnée manquante ou mal typée
401	Unauthorized	Authentification absente
403	Forbidden	Authentifié mais accès refusé
404	Not Found	Ressource introuvable
409	Conflict	Conflit avec l'état actuel (chevauchement, doublon)

5.4 Structurer une réponse d'erreur cohérente

Quel que soit le framework choisi, toutes les erreurs de votre API doivent suivre le même format JSON :

```
{  
  "erreur": "Conflit de réservation",  
  "detail": "La salle 1 est déjà réservée sur ce créneau"  
}
```

5.5 Erreurs fréquentes à éviter

- Toujours répondre 200 même en cas d'erreur métier (le client ne peut alors plus réagir correctement)
- Mélanger les conventions de nommage (snake_case et camelCase) dans une même API
- Oublier de vérifier l'existence de la ressource parente avant de créer une sous-ressource
- Valider uniquement côté client et faire confiance aux données reçues côté serveur

6. Critères d'évaluation

6.1 Exercice 1 — Benchmark Flask vs FastAPI

Critère	Points	Description
Parité fonctionnelle	/4	Les deux versions exposent exactement les mêmes endpoints
Validation des données	/4	Champs obligatoires vérifiés dans les deux versions
Tableau comparatif rempli	/4	Observations concrètes et justifiées, pas de réponse générique
Test de l'email mal formé	/3	Comparaison correcte du comportement des deux frameworks

6.2 Exercice 2 — Architecture RESTful

Critère	Points	Description
Conception des ressources	/4	Réponses aux questions de modélisation cohérentes et justifiées
Nommage des routes	/3	URLs conformes aux conventions REST (noms, pluriel, imbrication)
Gestion du chevauchement	/5	Détection correcte des conflits de créneaux, code 409
Codes de statut	/4	Tous les cas du tableau 4.5 correctement gérés
Tests pytest	/4	Couverture des cas nominaux et des cas d'erreur, y compris 409

7. Annexes

7.1 Ressources complémentaires

- Documentation Flask — <https://flask.palletsprojects.com/>
- Documentation FastAPI — <https://fastapi.tiangolo.com/>
- Documentation Pydantic — <https://docs.pydantic.dev/>
- Liste complète des codes de statut HTTP — <https://developer.mozilla.org/fr/docs/Web/HTTP/Status>

7.2 Commandes de référence rapide

```
# Lancer la version Flask
cd td2_benchmark/version_flask && python app.py

# Lancer la version FastAPI
cd td2_benchmark/version_fastapi && uvicorn app:app --reload --port 8000

# Lancer les tests de l'exercice 2
cd td2_architecture && pytest tests/ -v
```

7.3 Troubleshooting

Problème	Solution
pydantic.error_wrappers.ValidationError non capturée	FastAPI la transforme automatiquement en 422 ; ce n'est pas une erreur à corriger
Port déjà utilisé entre les deux versions	Garder Flask sur 5000 et FastAPI sur 8000 pour les lancer simultanément
EmailStr lève une erreur d'import	Installer la dépendance optionnelle : <code>pip install pydantic[email]</code>